تصميم البرمجيات

(Software Design)

المحاضرة (3)

- **مفاهيم تصميم البرمجيات.**
  - ❖ **تقنيات التمكين.**

- **مفاهيم تصميم البرمجيات:**
  - **تقنيات التمكين:**

هي: مفاهيم أساسية للعديد من مناهج تصميم البرمجيات.

- **التجريد:**

هو مبدأ أساسي يسمح بالتركيز على الجوانب المهمة وتجاهل التفاصيل غير الضرورية. ويساعد على تبسيط تصميم البرمجيات ويجعلها أكثر فهما وقابلية للإدارة. هناك عدة أنواع رئيسية للتجريد في تصميم البرمجيات:

1. **التجريد الوظيفي:** يركز على الوظائف والخدمات التي يقدمها النظام بدلاً من التركيز على التنفيذ التقني. هذا النوع من التجريد يسمح بإخفاء التفاصيل الداخلية للنظام.
2. **التجريد الهيكلي:** يتعلق بتقسيم النظام إلى مكونات متداخلة ومستقلة. هذا النوع من التجريد يساعد على تبسيط بنية النظام وتحسين قابليته للصيانة والتوسع.
3. **التجريد الواجهي:** يركز على فصل الواجهة المنطقية للنظام عن التنفيذ الداخلي. هذا يسمح بتغيير التنفيذ الداخلي دون التأثير على الواجهات المعروضة للمستخدمين.
4. **التجريد التكنولوجي:** يسمح باستبدال التقنيات المستخدمة في النظام دون إعادة تصميم النظام بأكمله. هذا يزيد من مرونة النظام وقابليته للتكيف مع التطورات التكنولوجية.

باختصار، التجريد هو أداة قوية في تصميم البرمجيات تساعد على تبسيط التصميم وزيادة قابليته للإدارة والصيانة والتوسع.

- **اقتران وتماسك :**

1. الاقتران:(Coupling)

- الاقتران يشير إلى درجة التعتيم أو التداخل بين مكونات النظام البرمجي.
- هناك أنواع مختلفة من الاقتران، تتراوح من الاقتران الضعيف (منخفض) إلى الاقتران القوي (مرتفع).
- الاقتران الضعيف هو المرغوب فيه، حيث يعني أن المكونات منفصلة بشكل جيد وتعتمد على بعضها البعض بأقل قدر ممكن.
- الاقتران القوي يجعل المكونات متداخلة بشكل كبير، مما يؤدي إلى صعوبة في الصيانة والتعديل والإعادة استخدام.

2. التماسك:(Cohesion)

- التماسك يشير إلى درجة ترابط وتماسك وحدات الكود داخل مكون واحد.
- هناك أنواع مختلفة من التماسك، تتراوح من التماسك المنخفض إلى التماسك المرتفع.
- التماسك المرتفع هو المرغوب ، حيث يعني أن المكون له وظيفة واضحة ومحددة، وأن جميع الوحدات داخله مرتبطة بهذه الوظيفة.
- التماسك المنخفض يعني أن المكون يؤدي مجموعة متنوعة من المهام غير ذات الصلة ببعضها البعض، مما يجعله صعب الصيانة والتعديل.

في التصميم الجيد للبرمجيات ، يجب السعي إلى تحقيق اقتران ضعيف وتماسك مرتفع، مما يؤدي إلى نظام قابل للصيانة وإعادة الاستخدام.

- **التحلل والنمذجة:**

1. التحلل:(Decomposition)

- التحلل هو عملية تقسيم المشكلة المعقدة إلى مجموعة من المشاكل الفرعية الأبسط والأسهل في التنفيذ.
- يساعد التحلل على فهم المشكلة بشكل أفضل وتبسيط عملية التصميم والتنفيذ.
- يتم تطبيق التحلل على مستويات مختلفة، مثل تقسيم النظام إلى مكونات رئيسية، ثم تقسيم كل مكون إلى وحدات أصغر.
- التحلل الجيد يؤدي إلى تصميم برمجي منظم ومرن وسهل الصيانة.

2. النمذجة:(Modeling)

- النمذجة تشير إلى تمثيل المشكلة أو النظام باستخدام نماذج مجردة، مثل الرسومات البيانية أو المخططات.
- النمذجة تساعد على توضيح الهيكل والعلاقات بين مكونات النظام قبل البدء في التنفيذ.
- توجد أنواع مختلفة من النماذج، مثل نماذج الأنشطة، نماذج البيانات، نماذج التفاعل، نماذج المكونات، إلخ.
- النمذجة الجيدة تساعد على اكتشاف المشاكل والمتطلبات المفقودة في مرحلة التصميم، مما يؤدي إلى تقليل التكاليف والوقت في مراحل لاحقة.

يعتبر التحلل والنمذجة عمليتين مترابطتين ومكملتين لبعضهما البعض. إذ يساعد التحلل على فهم المشكلة بشكل أفضل، بينما تساعد النمذجة على توضيح التصميم والعلاقات بين مكونات النظام قبل البدء في التنفيذ.

- **التغليف / إخفاء المعلومات:**

1. التغليف:(Encapsulation)

- التغليف هو أحد المبادئ الأساسية في التصميم للبرمجة الشيئية (OOP) (Object-Oriented Programming)
- التغليف يعني إخفاء التفاصيل الداخلية لكائن ما وتعريض واجهة محددة للتفاعل معه.
- يساعد التغليف على تحقيق مبدأ "المسؤولية الواحدة " حيث يضمن أن كل كائن مسؤول عن مهمة محددة.
- التغليف يجعل الكود أكثر قابلية للصيانة والتطوير، حيث يمكن تعديل التفاصيل الداخلية للكائن دون التأثير على الكود المستخدم له.

2. إخفاء المعلومات:(Information Hiding)

- إخفاء المعلومات هو مبدأ يسعى إلى الحد من الوصول إلى المعلومات الداخلية لكائن ما.
- يتم تحقيق إخفاء المعلومات من خلال تعريض واجهة محدودة للتفاعل مع الكائن، مع إخفاء التفاصيل الداخلية.
- يساعد إخفاء المعلومات على تقليل التبعيات بين الكائنات وتحسين قابلية الصيانة والتطوير.
- يمكن تطبيق إخفاء المعلومات على مستوى الكائنات والوظائف، أو المكونات الأخرى في البرنامج.

يعتبر التغليف وإخفاء المعلومات مبدأين مترابطين ومهمين لتحقيق مرونة الكود وقابليته للصيانة والتطوير. التغليف يساعد على تنظيم الكود والحد من التعقيد، بينما إخفاء المعلومات يحد من الاعتماد المفرط على مختلف مكونات البرنامج.

- **فصل الواجهة عن التنفيذ:**

- الواجهة تحدد الخصائص والسلوكيات المعروضة للعميل أو المستخدم، بينما التنفيذ يحتوي على التفاصيل الداخلية لتحقيق هذه الخصائص والسلوكيات.
- يسمح هذا الفصل بإجراء تغييرات في التنفيذ دون التأثير على الواجهة، والعكس صحيح.

فوائد فصل الواجهة عن التنفيذ:

- زيادة مرونة التصميم: يمكن تغيير التنفيذ دون الحاجة لتعديل الواجهة.
- تسهيل الاختبار والصيانة: يمكن اختبار الواجهة بشكل مستقل عن التنفيذ.
- تحسين إعادة الاستخدام: يمكن استخدام نفس الواجهة مع تنفيذات مختلفة.
- تبسيط التطوير: يمكن للمطورين التركيز على تطوير الواجهة والتنفيذ بشكل منفصل.

تطبيقات فصل الواجهة عن التنفيذ:

- في البرمجة الشيئية (OOP) يتم فصل الواجهة عن التنفيذ باستخدام الفصل بين الفئات المجردة والفئات الملموسة.
- في برمجة الخدمات (API) يتم فصل الواجهة عن التنفيذ الداخلي للخدمة. حيث أن واجهة برمجة التطبيقات (Application Programing Interface) تسمح للبرامج والتطبيقات المختلفة بالتفاعل والتواصل فيما بينها
- في برمجة الواجهات الرسومية، يتم فصل الواجهة المرئية عن المنطق الداخلي للتطبيق.

فصل الواجهة عن التنفيذ هو مبدأ أساسي في تصميم البرمجيات الجيد، حيث يساعد على زيادة المرونة والقابلية للصيانة والتطوير في المشاريع البرمجية.

- **الكفاية والاكتمال:**

1. الكفاية:(Sufficiency)

- الكفاية تعني أن التصميم البرمجي يوفر جميع الميزات والوظائف اللازمة لتحقيق متطلبات المستخدم.
- يجب أن يشتمل التصميم على كل ما هو ضروري لإنجاز المهام المطلوبة دون زيادة غير ضرورية.
- تحقيق الكفاية يساعد على تجنب التعقيد الزائد والحفاظ على تصميم بسيط وموجه نحو الهدف.

2. الاكتمال:(Completeness)

- الاكتمال يعني أن التصميم البرمجي يغطي جميع الحالات المحتملة والسيناريوهات المتوقعة.
- يجب أن يتعامل التصميم بشكل صحيح مع جميع المدخلات الممكنة والأخطاء المحتملة.
- تحقيق الاكتمال يضمن أن البرنامج سيعمل بشكل موثوق في جميع الظروف المتوقعة.

3. البدائية:(Primitiveness)

- البدائية تعني أن التصميم البرمجي يعتمد على مفاهيم وعمليات أساسية وبسيطة.
- يجب أن يتجنب التصميم التعقيد غير الضروري واستخدام مفاهيم مركبة أو معقدة بدون حاجة.
- تحقيق البدائية يساعد على جعل التصميم أكثر وضوحًا وسهل الفهم والصيانة.

في مجمل الأمر، الكفاية والاكتمال والبدائية هي مبادئ تصميم برمجي هامة تساعد على إنشاء تصاميم فعالة وموثوقة وسهلة الفهم. التوازن بين هذه المفاهيم هو جزء أساسي من عملية تصميم البرمجيات الجيد.